DESARROLLO DE APLICACIONES II

Respuesta a M3 Obras Públicas — conexión y registro

Módulo 9 · Core

13 de octubre de 2026

Contenido

1. `occurredAt` o `occuredAt` 3
2. URL de Core para Test y Producción 3
3. Datos de conexión a RabbitMQ 3
4. Secreto de máquina de `obras` 4
5. Exchange y cola 4
6. Publicaciones y suscripciones registradas para Obras 4
7. Endpoints de auditoría, dashboard y DLQ 5
8. Registrar o actualizar eventos y JSON Schema 6
9. Aislamiento entre Test y Producción 7
Mientras tanto: pueden integrar hoy 7
Lo que necesitamos de ustedes 7

Resumen: de los 9 puntos, **5 los contestamos ahora y cerrados** (1, 5, 6, 7, 8). Los otros 4 (2, 3, 4, 9) dependen de un deploy que **todavía no existe** — no hay entorno de Test ni de Producción levantado, ni nuestro ni de RabbitMQ. No se lo podemos inventar, así que abajo decimos qué falta y qué pueden hacer mientras tanto.

Además encontramos **un problema real en la integración de ustedes** que hoy haría que un evento que esperan nunca llegue. Está en el punto 6.

---

1. `occurredAt` o `occuredAt`

`occurredAt`, con doble `r`. El que están usando provisionalmente es el correcto.

No es una preferencia de estilo: el sobre declara `additionalProperties: false`, así que la variante con una sola `r` no se ignora, **se rechaza**. Pasamos las dos por el validador real:

```
occurredAt   → ACEPTADO
occuredAt    → RECHAZADO [missing]          campo occurredAt: Field required
               RECHAZADO [extra_forbidden]  campo occuredAt: Extra inputs are not permitted
```

Y un detalle que les va a ahorrar un rato: **el offset de zona horaria es obligatorio.**

```
"2026-10-13T12:00:00-03:00"  → ACEPTADO
"2026-10-13T12:00:00"        → RECHAZADO: occurredAt debe incluir zona horaria
```

Un timestamp sin offset es ambiguo entre 9 módulos desplegados por separado, así que preferimos rechazarlo antes que asumir la hora local del servidor.

---

2. URL de Core para Test y Producción

No existen todavía. El Core no está desplegado en ningún lado: hoy corre sólo en las máquinas de desarrollo, en `http://localhost:8000`.

No les damos una URL que no vamos a poder sostener. Cuando el equipo decida dónde desplegar, se las pasamos con los dos entornos. Mientras tanto pueden integrar contra su propia instancia local (ver el final de este documento).

---

3. Datos de conexión a RabbitMQ

Tampoco existen todavía, por lo mismo: no hay un RabbitMQ compartido levantado. Hoy cada uno corre el suyo localmente, con la configuración de desarrollo de siempre:

	Valor local
Host / puerto	`localhost:5672`
TLS	no (en el deploy sí, con `amqps://`)
Virtual host	`/`
Usuario	`guest` / `guest`
Consola	`http://localhost:15672`

Cuando haya broker compartido les pasamos host, puerto, vhost y **un usuario por módulo** (no el `guest`), con permisos acotados a lo que necesitan: publicar en `muni.inbox` y consumir de `q.obras`.

---

4. Secreto de máquina de `obras`

Dos cosas:

No lo tenemos para darles. El Core guarda el secreto como hash SHA-256 (64 caracteres), nunca en claro. Se imprime una sola vez, cuando se genera. Si se perdió, no se recupera: se rota y se emite uno nuevo con `POST /api/v1/modules/obras/rotate-secret`.

No se manda por acá. Un secreto de máquina pegado en el chat del grupo queda en el historial de todos para siempre. Cuando exista el deploy se los pasamos por un canal privado, uno a uno. Rotarlo no les corta el acceso al dashboard a nadie: son credenciales separadas a propósito.

---

5. Exchange y cola

Confirmado. Los dos nombres que usan son los correctos.

	Nombre	Tipo
Donde publican	`muni.inbox`	fanout
De donde consumen	`q.obras`	durable

El recorrido completo, por si les sirve para el diagrama:

```
obras ──► muni.inbox ──► core.inbox ──► [el Core valida, guarda y rutea]
                                                  │
                                                  ▼
                                          muni.events  (topic)
                                                  │  routing key = nombre de la cola
                                                  ▼
                                             q.obras
```

Dos cosas que se desprenden de eso:

- **No se bindean ustedes a `muni.events`.** El Core declara la cola y el binding cuando se registra la suscripción. Ustedes sólo consumen de `q.obras`.
- **Si el Core está caído, igual pueden publicar.** `core.inbox` es durable y hace de buffer; cuando el Core vuelve, drena la cola. No pierden eventos.

---

6. Publicaciones y suscripciones registradas para Obras

Esto es lo que tiene el catálogo del Core hoy, con quién está del otro lado.

Publican (6)

Evento	Lo escucha
`publicWorksProjectApproved`	ciudadanos
`publicWorksProjectCompleted`	ciudadanos
`streetClosureRequested`	transito
`updateTicketStatus`	atencion-ciudadana
`workOrderCompleted`	ambiente, transito
`workOrderScheduled`	ambiente, transito

Escuchan (10)

Evento	Lo publica
`caseFileResolved`	ciudadanos
`containerDamaged`	ambiente
`infrastructureRepairRequested`	ambiente
`roadAccidentRegistered`	transito
`streetClosureApproved`	transito
`streetClosureEnded`	**nadie — ver abajo**
`streetClosureRejected`	transito
`ticketCreated`	atencion-ciudadana
`ticketUpdated`	atencion-ciudadana
`treeRiskDetected`	ambiente

Todas activas, con 4 reintentos.

⚠️ `streetClosureEnded` nunca les va a llegar

Ustedes están suscriptos a **`streetClosureEnded`**. Tránsito declaró que publica **`streetClousureEnded`** — con una `u` de más.

```
streetClosureEnded     publica: nadie    escucha: obras
streetClousureEnded    publica: transito escucha: nadie
```

Son dos tipos distintos para el hub. Tal como está hoy, ese evento sale de Tránsito y no llega a nadie, y ustedes esperan algo que nunca va a venir.

Hay que ponerse de acuerdo con Tránsito en un solo nombre (nosotros sugerimos `streetClosureEnded`, que es el que está bien escrito) y que el que cambie rehaga su declaración. Avisamos a Tránsito también.

Esto no lo encontramos a mano: lo detecta solo el Core cruzando quién publica qué con quién consume qué, y lo expone en `GET /api/v1/dashboard/integration-alerts`. Si quieren chequear su propia integración antes de cada entrega, es el endpoint.

---

7. Endpoints de auditoría, dashboard y DLQ

Todos bajo `/api/v1`, con `Authorization: Bearer <token>`.

Auditoría

Endpoint	Qué devuelve
`GET /events`	Su bitácora: lo que publicaron y lo que les entregamos. Filtros: `query`, `eventType`, `sourceModule`, `status`, `correlationId`, `since`, `until`, `page`, `size`.
`GET /events/{eventId}`	El sobre completo más el estado de **cada entrega**, módulo por módulo.
`GET /events/journey/{correlationId}`	El recorrido del trámite completo cruzando los 9 módulos.
`GET /deliveries`	Entregas con su estado y número de intento.

Dashboard

Endpoint	Qué devuelve
`GET /dashboard`	Su tablero: publicados, recibidos, dead letters, suscripciones, volumen por hora.
`GET /dashboard/integration-alerts`	**Los agujeros de integración.** Es el del punto 6.

DLQ

Endpoint	Permiso
`GET /dlq`	Lo ven ustedes
`GET /dlq/{id}`	Lo ven ustedes
`GET /dlq/{id}/audit`	Lo ven ustedes
`POST /dlq/{id}/retry`	Sólo admin (equipo 9)
`POST /dlq/retry-bulk`	Sólo admin
`POST /dlq/{id}/discard`	Sólo admin

Importante sobre el alcance: el Core filtra por el módulo del token. Ustedes ven su tráfico y nada del ajeno; no hace falta que filtren del lado de ustedes.

---

8. Registrar o actualizar eventos y JSON Schema

Con el token de su módulo, no hace falta pedirnos nada.

Declarar un tipo de evento

```
curl -X POST http://<core>/api/v1/event-types \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"workOrderScheduled","description":"Se programó una orden de trabajo"}'
```

Declarar que lo publican → `POST /publications` **Suscribirse a uno ajeno →** `POST /subscriptions` (el Core declara la cola y el binding en el acto)

Cargar el JSON Schema de un tipo

```
curl -X PUT http://<core>/api/v1/event-types/workOrderScheduled/schema \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"jsonSchema":{"type":"object","required":["workOrderId"],
       "properties":{"workOrderId":{"type":"string"}}}}'
```

Tres cosas que conviene saber:

- **El schema es opcional y por tipo.** Sin schema el `data` pasa sin que lo miren; con schema se valida la estructura y lo que no cumple va a la DLQ con el campo exacto que falló. Actívenlo cuando estén listos: no frena a nadie.
- **Un tipo no declarado no se rechaza.** Si publican algo que nadie registró, el Core lo da de alta solo y lo marca como `discovered`. Preferimos eso a trabar la integración por un typo — pero después aparece en las alertas.
- **Hoy cualquier módulo autenticado puede escribir el schema de cualquier tipo.** No hay chequeo de propietario. Lo decimos para que no cuenten con esa barrera: por ahora es coordinación entre equipos, no una restricción técnica.

---

9. Aislamiento entre Test y Producción

Hoy no hay nada que aislar, porque no hay entornos.

La forma en que está construido permite separarlos del todo —son dos variables de entorno, `DATABASE_URL` y `RABBITMQ_URL`, y ningún nombre de exchange o cola está escrito en el código— pero eso es una propiedad del diseño, no un hecho. Prometerles aislamiento antes de tener los dos entornos levantados sería prometer algo que no podemos demostrar.

Cuando se despliegue, la separación que proponemos es:

	Test	Producción
Base	instancia propia	instancia propia
RabbitMQ	**vhost propio**	vhost propio
Secretos	distintos	distintos
Clave de firma JWT	distinta	distinta

Vhosts separados y no prefijos de cola: con prefijos, un error de configuración publica en producción sin que nada falle. Con vhosts, las credenciales de Test sencillamente no pueden ver producción.

Mientras tanto: pueden integrar hoy

No hace falta esperar al deploy. El Core levanta local en dos comandos y van a poder probar el camino completo contra su propio RabbitMQ:

```
git clone <repo> && cd backend
.venv/bin/python -m app.seeds          # carga los 9 módulos y los 73 tipos
.venv/bin/uvicorn app.main:app --reload
```

- Swagger navegable en `http://localhost:8000/docs`
- Usuario de su módulo: `obras@muni.uade.edu.ar` / `Cambiala123`
- El worker que consume de RabbitMQ va aparte:
  `.venv/bin/python -m app.workers.inbox_worker`

El seed imprime el secreto de máquina de cada módulo **la primera vez**.

Lo que necesitamos de ustedes

1. **Ponerse de acuerdo con Tránsito sobre `streetClosureEnded`** (punto 6). Es lo único que hoy está roto de verdad.
2. Confirmar que la lista de 6 publicaciones y 10 suscripciones es la definitiva. Si falta o sobra algo, lo ajustan ustedes desde el panel.

Y lo que falta de nuestro lado —puntos 2, 3, 4 y 9— se destraba todo junto el día que el equipo decida dónde desplegar.